

Search in Unknown Environments

Searching in an unknown environment involves agents (robots, software, animals) using local sensor data and experience to explore, map, and find targets without prior global knowledge, employing strategies like online learning, exploration/pursuit states, and sometimes multi-agent cooperation, dealing with uncertainty, obstacles, and limited "lifespans" (fuel, computation). Key techniques include real-time search, reinforcement learning, and area coverage algorithms to adapt to dynamic settings and find goals efficiently.

Common Strategies & Algorithms

Online Search: Agents act, observe, learn, and then decide the next move, gradually building knowledge.

Real-Time Search (RTS): Methods like LRTA* gather early information to resolve uncertainty.

Exploration/Exploitation: Balancing discovering new areas (exploration) with using known good paths (exploitation).

State-Based AI: Using states like "Exploration," "Pursuit," and "Predictive Tracking" (for moving targets).

Fuzzy Logic & Template Matching: For path planning and target recognition with sensors (cameras, sonar).

Multi-Agent Systems: Teams of robots share info and divide tasks (e.g., UAVs for area coverage).

Examples in Practice

- **Robotics:** Search & rescue, environmental monitoring, autonomous exploration.
- **Computer Science:** AI agents in games, network routing, large database searches.
- **Biology:** Animal foraging for food or mates.

1. Core Problem Overview

An agent must operate in an environment where:

- **No prior map:** The agent must build its understanding as it goes.
- The **locations of obstacles, free space, hazards, and goal are unknown.**
- Observations are **local**, not global.
- Actions must be chosen despite **uncertainty**(Outcomes of actions aren't fully predictable.).
- **Finite resources:** Agents have limited time, fuel, or processing power.
- **Obstacles:** Physical barriers hinder movement.

The challenge is to:

Explore + map + plan simultaneously

(Unlike classical search where the full graph/map is known.)

This is the foundational problem in robotics, RL, autonomous navigation, Real-time games, and embodied AI.

2. Core Concepts

(a) Belief / Knowledge Representation

Agent stores:

- **Known free cells**
- **Known obstacles**
- **Frontier cells** (boundary between known & unknown)
- **Unknown region** to be explored

Usually represented as an **occupancy grid**.

(b) Exploration Strategy

Frontier-Based Exploration

- Find cells adjacent to unknown space → explore them.

Process:

1. Find all frontier cells.
2. Select a frontier.
3. Plan path to it using A* or D* Lite.
4. Move, update map, repeat.

Strength:

- Simple, efficient, widely used in robotics.

D*, D* Lite

- Repair A* paths when discovering new obstacles.
 - Instead of re-running A* from scratch when obstacles appear,
 - Repair the existing search tree efficiently.

LRTA*(Learning Real-Time A*)

- Learn heuristics incrementally as the agent moves.
- Must improve their internal heuristic model over time, move while planning
 - Agent cannot compute full plan (unknown map).
 - It only plans locally.
 - After every move, it updates the heuristic:

$$h(s) \leftarrow 1 + \min(h(\text{neighbors}))$$

- Over multiple traversals, LRTA* learns near-optimal paths(Guarantees convergence to optimal paths after repeated trials).

Reinforcement Learning Exploration Strategy

Agents trained with RL use:

- Random exploration (ϵ -greedy)
- Entropy regularization
- Curiosity (internal reward for discovering new states)
- Random Network Distillation (novelty detection)

These methods let RL agents explore unknown terrain efficiently.

(c) Information-Gain Planning

Agent selects actions not only to reach a goal, but to:

- Reduce uncertainty
- Map unexplored areas
- Avoid getting stuck

(d) SLAM (Simultaneous Localization and Mapping)

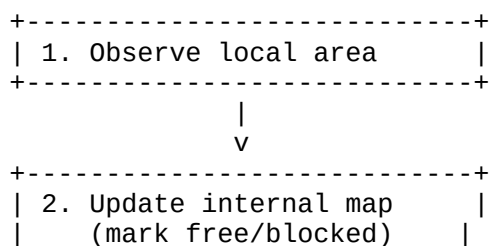
SLAM + Planning = Intelligent Agents

Agent:

1. Tracks its own position (localization)
 - Kalman filters
 - Particle filters
 - Scan-matching
2. Builds a map (mapping)
 - Occupancy grid
 - Probabilistic maps
3. Plans through the map (planning)
 - A*, D*, RRT*
 - MPC(Model Predictive Control)
 - RL-based planners

SIMPLE IMPLEMENTATION LOGIC

This is the typical cycle for an agent searching an unknown environment:



1. Is **free**, and
2. Is adjacent to **unknown** cells

Frontiers represent “*edges of the known world.*”

4. Frontier Selection

Heuristics:

- Nearest frontier
- Frontier with maximum unknown area behind it
- Frontier with highest expected information gain

5. Path Planning

Use A* initially.

As you discover new obstacles:

- Switch to *D Lite** to avoid full re-planning.

Path is planned from:

Current location → chosen frontier

6. Movement + Replanning

For each step:

- Move one cell along planned path
- Re-scan sensors
- Update map
- If path becomes invalid → re-plan

7. Goal Detection

If the goal becomes visible (e.g., object, target location), then:

1. Switch from **exploration mode** to **goal-directed mode**
2. Run A*/D* to the goal

DIAGRAMS

1. Occupancy Grid in Unknown Environment

Legend:

. = free

= obstacle

? = unknown
A = agent

Initial state:

```
?????????  
?????????  
?????A????  
?????????  
?????????
```

After some steps:

```
#####?????  
#...#?????  
#..A#?????  
#...#?????  
#####?????
```

Agent sees only its **local region**, expanding knowledge outward.

2. Frontier Identification

Frontiers = free cells adjacent to unknown cells

Known map:

```
#####???  
#..A???  
#.....?  
#####
```

Frontiers (F):

```
#####F??  
#..AFF?  
#....F?  
#####
```

The agent selects one to explore next.

3. Path Planning With Incremental Replanning

Before discovering new obstacle:

```
A → → → Goal  
. . . . G
```

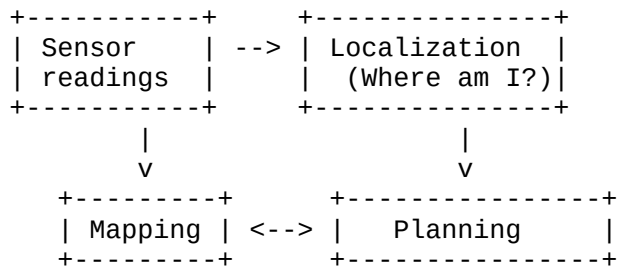
Obstacle discovered during movement:

```
A → X   G  
. . # . .
```

Agent **updates map**, then **repairs plan** using D* Lite:

```
A → ↓ → G  
. . # . .  
. . . . .
```

4. SLAM Loop (Conceptual)



Mapping updates → planning updates → new actions → new observations.

ULTRA-SHORT SUMMARY

SEARCH IN UNKNOWN ENVIRONMENTS =

Observe → Update map → Detect frontiers → Choose best frontier →

Plan path → Move → Replan when needed → Repeat until goal found

Algorithms involved:

- Frontier-based exploration
- A*, LRTA*, D*, D* Lite
- SLAM (mapping + localization + planning loop)
- Uncertainty-driven exploration (POMDP concepts)